# Laborator 3: Variabile. Tipuri de date. Operatori.




## Cuprins

*1 Obiective și concepte fundamentale* 2
*2 Variabile și obiecte* 2
*3 Declarații, definiții și inițializare* 2
*4 Tipuri fundamentale de date* 4
*5 Literale și reprezentarea valorilor* 4
*6 Operatori și expresii* 5
*7 Operatori relaționali, de egalitate și logici* 6
*8 Operatori de incrementare și decrementare* 7
*9 Operatori pe biți și de deplasare* 7
*10 Operatorul condițional* 8
*11 Precedența, asociativitatea și conversia de tip* 8
*12 Operatorul `sizeof`, `auto` și verificarea tipurilor* 10
*13 Exemple de analiză a expresiilor* 10
*14 Aplicații* 11

# 1. Obiective și concepte fundamentale

**Obiectivele laboratorului**

La finalul laboratorului studentul trebuie să poată:

- declare, definească și inițializeze variabile în C++;
- identifice tipul potrivit pentru o anumită categorie de date;
- distingă între tipuri întregi, reale, logice și caractere;
- utilizeze specificatorii `signed`, `unsigned`, `short`, `long`;
- determine dimensiunea unei variabile sau a unui tip cu `sizeof`;
- recunoască literalele întregi, reale, caracter și booleene;
- utilizeze operatori aritmetici, relaționali, de egalitate, logici și pe biți;
- înțeleagă precedența și asociativitatea operatorilor;
- utilizeze operatorii de atribuire, incrementare/decrementare și condițional;
- distingă între conversiile implicite și conversiile explicite de tip;
- analizeze expresii C++ și să anticipeze rezultatul evaluării lor.

# 2. Variabile și obiecte

O **variabilă** este o zonă de memorie în care este stocată o informație. Ea are asociate un nume și un tip de date. Tipul determină dimensiunea memoriei, modul de interpretare a informației și operațiile care pot fi aplicate.

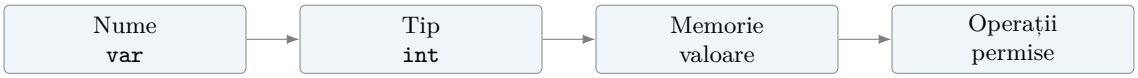



**Figura 1:** Relația dintre numele unei variabile, tipul său și memoria utilizată.

## 2.1. Identificatori

Un identificator este un nume format din litere, cifre și caracterul `_`, cu condiția ca primul caracter să nu fie o cifră. C++ este **case sensitive**: `valoare`, `Valoare` și `VALOARE` sunt identificatori diferiți.

```
int var1;
int suma_totala;
double temperatura;
bool esteValid;
```

Nu trebuie folosite cuvinte cheie precum `int`, `for`, `while`, `return` sau `class` ca nume de variabile.

**Alegerea numelor**

Este preferabil un nume care descrie rolul variabilei:

```
double temperatura_celsius;
int numar_studenti;
bool este_prim;
```

în locul unor nume fără semnificație precum `x1`, `a2`, `q`.

# 3. Declarații, definiții și inițializare

O declarație anunță existența unui nume. Pentru variabile, forma de bază este:

```
tip_de_date nume;
```

În multe cazuri, o declarație de variabilă este simultan și o definiție, deoarece se stabilește și spațiul de memorie necesar.

**Declarație și inițializare**

```
int a;              // definit, valoare neinitializata
int b = 10;         // initializare
double x = 3.14;
char c = 'A';
bool ok = true;
```

## 3.1. Atribuirea

Atribuirea folosește operatorul `=`

```
int a = 5;
a = 12;
```

După a doua instrucțiune, valoarea lui `a` este 12.

Operatorul de atribuire este diferit de operatorul de egalitate `==`. Confuzia dintre cele două este o sursă frecventă de erori.

## 3.2. Durata de viață și domeniul de vizibilitate

Variabilele locale sunt asociate blocului în care sunt declarate. Un bloc este delimitat de `{` și `}`.

```
int main()
{
    int a = 5;

    {
        int b = 10;
        cout << a + b << endl;
    }

    // b nu mai poate fi folosit aici
    return 0;
}
```

**Regulă importantă**

O variabilă poate fi utilizată numai în zona în care numele său este vizibil. Pentru variabilele locale, aceasta este în principal zona blocului în care au fost declarate și blocurile imbricate din interiorul acestuia.

## 3.3. Constante

O valoare literală este o valoare fixă scrisă direct în program:

- `42` – literal întreg;
- `3.14` – literal real;
- `'A'` – literal caracter;
- `true`, `false` – literale booleene;
- `"Salut"` – literal șir de caractere.

O variabilă constantă este diferită de un literal:

```
const double PI = 3.141592653589793;
```

După inițializare, valoarea lui `PI` nu poate fi modificată.

# 4. Tipuri fundamentale de date

Tipul unei variabile stabilește dimensiunea memoriei, domeniul de valori, interpretarea informației și operațiile care pot fi aplicate.

| **Tip** | Dimensiune | Interval de definiție | Utilizare |
|---|---|---|---|
| `(unsigned) char` | 1 byte | $[-128, 127]$ sau $[0, 255]$ | caractere și valori întregi mici |
| `short int` | 2 bytes | $[-2^{15}, 2^{15} - 1]$ | întregi cu domeniu mai mic |
| `int` | 4 bytes | $[-2^{31}, 2^{31} - 1]$ | numere întregi generale |
| `unsigned int` | 4 bytes | $[0, 2^{32} - 1]$ | întregi nenegativi |
| `long long` | 8 bytes | $[-2^{63}, 2^{63} - 1]$ | numere întregi mari |
| `float` | 4 bytes | $[-3.4028 \cdot 10^{38}, 3.4028 \cdot 10^{38}]$ | numere reale, precizie simplă |
| `double` | 8 bytes | $[-1.7977 \cdot 10^{308}, 1.7977 \cdot 10^{308}]$ | numere reale, precizie dublă |
| `long double` | dependent-implementare | dependent-implementare | precizie reală extinsă |
| `bool` | dependent-implementare | $\{false, true\}$ | `true`/`false` |
| `void` | – | – | lipsa unei valori de tip |

**Tabela 1:** Tipuri fundamentale, dimensiunea și intervalul de definiție.

## 4.1. Tipuri întregi

Tipurile întregi includ `char`, `short`, `int`, `long` și `long long`. Specificatorul `unsigned` permite reprezentarea numai a valorilor nenegative și extinde domeniul pozitiv.

Pe sistemele de operare moderne, `int` este în mod uzual pe 32 de biți și poate reprezenta valori aproximativ în intervalul

$$[-2^{31}, 2^{31} - 1].$$

**Specificatori de tip**

```
short int a = 100;
unsigned int b = 4000000000u;
long long c = 90000000000000000LL;
```

Într-o situație în care domeniul valorilor contează, alegerea tipului trebuie făcută în funcție de limitele problemei.

## 4.2. Tipuri reale

`float`, `double` și `long double` reprezintă numere în virgulă mobilă. În mod uzual, `float` oferă aproximativ 7 cifre semnificative, iar `double` aproximativ 15 cifre semnificative.

## 4.3. Tipul `bool`

Tipul `bool` reprezintă o valoare logică:

```
bool activ = true;
bool gasit = false;
```

În expresii logice, zero este interpretat ca fals, iar o valoare diferită de zero este interpretată ca adevărat.

# 5. Literale și reprezentarea valorilor

### 5.1. Literale întregi

Constantele întregi pot fi scrise în mai multe baze:

```
int zecimal = 1245;
int octal = 01245;
int hexazecimal = 0x1245;
```

Prefixul `0` indică forma octală, iar `0x` sau `0X` forma hexazecimală.

Sufixurile pot modifica tipul literalului:

- `u` sau `U` – `unsigned`;
- `l` sau `L` – `long`;
- combinații precum `UL` sau `LL`.

### 5.2. Literale reale

Un literal real conține o parte fracționară și poate avea exponent:

```
3.14
-34.9e-2
2.5f
1.0L
```

În lipsa unui sufix, un literal real este de regulă de tip `double`. Sufixul `f` indică `float`, iar `L` poate indica `long double`.

### 5.3. Literale caracter

Caracterele sunt scrise între apostrofuri:

```
char a = 'A';
char newline = '\n';
char tab = '\t';
```

Caracterele speciale sunt reprezentate prin secvențe escape. Câteva exemple:

| Secvență | Semnificație |
|---|---|
| `\n` | linie nouă |
| `\t` | tabulator |
| `\\` | caracterul backslash |
| `\'` | apostrof |
| `\"` | ghilimele |

**Tabela 2:** Exemple de secvențe escape.

În implementările uzuale, caracterele ASCII sunt asociate unor valori întregi. De exemplu, `'A'` are codul 65, iar `'a'` are codul 97.

### 5.4. Literale șir

Un șir de caractere este delimitat de ghilimele:

```
cout << "Programarea Calculatoarelor";
```

La nivel de reprezentare C / C++, un literal șir este încheiat cu caracterul `'\0'`.

## 6. Operatori și expresii

O expresie este alcătuită din operanzi și operatori. Operanzii pot fi constante, variabile sau apeluri de funcții, iar operatorii stabilesc modul în care aceștia sunt prelucrați.

În funcție de numărul de operanzi:

- **unari** – acționează asupra unui singur operand;
- **binari** – acționează asupra a doi operanzi;
- **ternari** – utilizează trei expresii, în cazul operatorului `?:`

### 6.1. Operatori aritmetici

Operatorii principali sunt:

| Operator | Semnificație | Exemplu |
|---|---|---|
| + | adunare | a+b |
| - | scădere / schimbarea semnului | a-b, -a |
| * | înmulțire | a*b |
| / | împărțire cu cât (DIV) | a/b |
| % | restul împărțirii întregi (MOD) | a%b |

**Tabela 3:** Operatori aritmetici.

**Împărțirea întreagă**

```
int a = 7, b = 2;
cout << a / b << endl;   // 3
cout << a % b << endl;   // 1
```

Dacă cel puțin unul dintre operanzi este real, rezultatul împărțirii este real:

```
double x = 7, y = 2;
cout << x / y << endl;   // 3.5
```

### 6.2. Operatori de atribuire

Operatorul simplu este =. Există și operatori de atribuire compusă:

| Operator | Echivalent |
|---|---|
| a += b | a = a + b |
| a -= b | a = a - b |
| a *= b | a = a * b |
| a /= b | a = a / b |
| a %= b | a = a % b |
| a «= b | a = a « b |
| a »= b | a = a » b |

**Tabela 4:** Câteva forme de atribuire compusă.

## 7. Operatori relaționali, de egalitate și logici

### 7.1. Operatori relaționali

Operatorii relaționali compară două expresii:

| Operator | Semnificație |
|---|---|
| < | mai mic |
| <= | mai mic sau egal |
| > | mai mare |
| >= | mai mare sau egal |

**Tabela 5:** Operatori relaționali.

Rezultatul este o valoare logică.

### 7.2. Operatori de egalitate

== verifică egalitatea, iar != verifică diferența:

```
int x = 5;
bool a = (x == 5);   // true
bool b = (x != 5);   // false
```

**Atenție: = versus ==**

`a = b` înseamnă atribuire, în timp ce `a == b` înseamnă comparație. Cele două simboluri nu sunt interschimbabile.

### 7.3. Operatori logici

Operatorii logici sunt:

| | |
|---|---|
| `!` | NOT – negare |
| `&&` | AND – și logic |
| `\|\|` | OR – sau logic |

**Tabela 6:** Operatori logici.

Exemplu:

```
int x = 7;
bool rezultat = (x > 0) && (x < 10);
```

### 7.4. Evaluarea cu scurtcircuit

Pentru `&&`, dacă primul operand este fals, al doilea nu mai este evaluat. Pentru `||`, dacă primul operand este adevărat, al doilea nu mai este evaluat.

**Exemplu**

```
int x = 0;
bool ok = (x != 0) && (10 / x > 1);
```

A doua expresie nu este evaluată deoarece prima este falsă. Acest mecanism poate fi folosit pentru a evita evaluarea unor expresii care ar produce probleme.

## 8. Operatori de incrementare și decrementare

`++` incrementează, iar `--` decrementează o variabilă.

```
int a = 10;
int x = a++;   // x = 10, apoi a = 11

int b = 10;
int y = ++b;   // b = 11, apoi y = 11
```

Diferența dintre forma prefixată și postfixată este importantă atunci când valoarea expresiei este folosită în aceeași instrucțiune.

## 9. Operatori pe biți și de deplasare

Operatorii pe biți se aplică tipurilor întregi și acționează asupra reprezentării binare.

## 9.1. Operatori bitwise

| Operator | Semnificație |
|---|---|
| ~ | complement față de 1 |
| & | AND pe biți |
| \| | OR pe biți |
| ^ | XOR pe biți |

**Tabela 7:** Operatori de prelucrare la nivel de bit.

**AND pe biți**

Pentru:

$$12 = 00001100_2, \qquad 5 = 00000101_2$$

rezultă:

$$12 \,\&\, 5 = 00000100_2 = 4.$$

În C++:

```
int a = 12;
int b = 5;
cout << (a & b);   // 4
```

## 9.2. Deplasări

Operatorii « și » deplasează biții:

```
int a = 3;
cout << (a << 4) << endl;   // 48
cout << (a >> 1) << endl;   // 1
```

Pentru valori pozitive, deplasarea la stânga cu $k$ poziții corespunde, atunci când nu se pierd biți, multiplicării cu $2^k$. Deplasarea la dreapta corespunde aproximativ împărțirii întregi la $2^k$.

## 9.3. Exemplu de mască

Operatorii pe biți sunt utilizați frecvent pentru testarea unor flag-uri:

```
int flags = 13;       // 1101
int mask  = 4;        // 0100

if ((flags & mask) != 0)
    cout << "Bitul este setat";
```

Parantezele sunt importante deoarece precedența operatorului `&` este diferită de cea a operatorului `==`.

# 10. Operatorul condițional

Operatorul ternar are forma:

```
conditie ? expresie1 : expresie2
```

Este echivalent cu alegerea dintre două expresii:

```
int a = 7, b = 10;
int maxim = (a > b) ? a : b;
```

Operatorul condițional este util pentru expresii scurte, dar pentru logică complexă este de preferat o structură `if`.

# 11. Precedența, asociativitatea și conversia de tip

## 11.1. Precedența operatorilor

Atunci când o expresie conține mai mulți operatori, aceștia sunt evaluați conform unor reguli de precedență. Parantezele pot modifica ordinea implicită.

| Nivel | Operatori | Exemplu |
|---|---|---|
| 1 | ++ - postfix | a++ |
| 2 | ++ - + - ! ~ unari | !ok |
| 3 | * / % | a*b+c |
| 4 | + - | a+b-c |
| 5 | « » | a«2 |
| 6 | < <= > >= | a<b |
| 7 | == != | a==b |
| 8 | & | a&b |
| 9 | ^ | a^b |
| 10 | \| | a\|b |
| 11 | && | a&&b |
| 12 | \|\| | a\|\|b |
| 13 | ?: | c?a:b |
| 14 | = += -= *= /= %= | a+=b |
| 15 | , | a,b |

**Tabela 8:** Ordine simplificat al precedenței operatorilor relevanți pentru laborator.

Operatorii unari, condiționalul și atribuirea se asociază de la dreapta la stânga; majoritatea operatorilor binari se asociază de la stânga la dreapta.

**De ce sunt utile parantezele?**

```
int x = 2 + 3 * 4;      // 14
int y = (2 + 3) * 4;    // 20
```

Parantezele fac intenția programatorului explicită și reduc riscul de erori.

## 11.2. Conversii implicite

Când într-o expresie apar tipuri diferite, C++ poate realiza automat conversii pentru a aduce operanzii la tipuri compatibile.

```
int a = 5;
double b = 2.0;
double c = a / b;    // a este convertit la double
```

În general, tipurile mai mici sunt promovate către tipuri capabile să reprezinte un domeniu mai larg. Caracterul este frecvent promovat la `int` în expresii aritmetice.

## 11.3. Conversii la atribuire

La atribuirea într-un tip mai mic poate apărea pierdere de informație:

```
double x = 3.75;
int y = x;       // y devine 3
```

Fracțiunea este eliminată în conversia către `int`.

## 11.4. Conversia explicită

Conversia explicită poate fi făcută cu operatorul de cast:

```
int a = 5;
int b = 2;

double x = (double)a / b;
```

# 12. Operatorul `sizeof`, `auto` și verificarea tipurilor

## 12.1. Operatorul `sizeof`

`sizeof` determină dimensiunea, în bytes, a unui tip sau obiect.

```
int a = 5;

cout << sizeof(a) << endl;
cout << sizeof(int) << endl;
cout << sizeof(double) << endl;
```

Dimensiunile exacte pot depinde de compilator, arhitectură și modelul de date. De aceea, atunci când dimensiunea contează, aceasta trebuie verificată cu `sizeof` și nu presupusă.

## 12.2. Tipul `auto`

C++ poate deduce automat tipul unei variabile inițializate:

```
auto x = 4;       // int
auto y = 3.37;    // double
auto ok = true;   // bool
auto c = 'A';     // char
```

O variabilă declarată cu `auto` trebuie inițializată la definire, deoarece compilatorul are nevoie de expresia inițială pentru a deduce tipul.

## 12.3. `decltype`

`decltype` poate fi folosit pentru a obține tipul unei expresii sau al unei variabile:

```
int x = 5;
decltype(x) y = 10;
```

În exemplu, `y` are același tip ca `x`.

# 13. Exemple de analiză a expresiilor

**Exemplul 1**

Determinați valorile:

```
int a = 10;
int b = 3;
int c = a / b;
int d = a % b;
```

Rezultatul este `c = 3` și `d = 1`.

**Exemplul 2**

Analizați:

```
int a = 10;
int x = a++;
int y = ++a;
```

După prima atribuire, `x` este 10 și `a` devine 11. Apoi `a` devine 12 înainte ca valoarea să fie atribuită lui `y`; deci `y = 12`.

**Exemplul 3**

Analizați:

```
int a = 5;
int b = 2;
double x = a / b;
double y = (double)a / b;
```

În primul caz, împărțirea se face ca împărțire întreagă și apoi rezultatul este convertit la `double`, deci `x = 2`. În al doilea caz, conversia are loc înaintea împărțirii, deci `y = 2.5`.

# 14. Aplicații

**Problema 1.** Scrieți un program care afișează `sizeof` pentru `char`, `short`, `int`, `long long`, `float`, `double` și `bool`. Comparați rezultatele obținute pe sistemul vostru cu valorile uzuale prezentate în laborator.

**Problema 2.** Declarați variabile pentru: vârsta unei persoane, media unui student, prima literă a numelui și existența unei condiții. Alegeți tipul potrivit și inițializați variabilele cu valori de exemplu. Afișați-le.

**Problema 3.** Citiți două numere întregi $a$ și $b$. Afișați câtul și restul împărțirii lor.
Ce se întâmplă în cazul în care $b = 0$.

**Problema 4.** Citiți un număr întreg și determinați dacă este par folosind operatorul modulo.

**Problema 5.** Citiți două numere întregi și verificați dacă primul este divizor al celui de-al doilea.
Hint: Folosiți operatorii `%` și `==`.

**Problema 6.** Citiți două numere reale și determinați maximul folosind exclusiv operatorul `?:` pentru alegerea rezultatului.

**Problema 7.** Fără a rula programul, determinați valorile afișate:

```
int a = 10;
cout << a++ << endl;
cout << ++a << endl;
cout << a-- << endl;
cout << --a << endl;
```

Verificați apoi rezultatul prin rulare.

**Problema 8.** Pentru `a = 12` și `b = 5`, calculați manual și apoi verificați în C++ rezultatele pentru:

$$a \,\&\, b, \quad a|b, \quad a \,\hat{}\, b, \quad a \ll 1, \quad b \gg 1.$$

**Problema 9.** Citiți un număr întreg și o poziție de bit $k$. Verificați dacă bitul $k$ este setat folosind o mască de forma $1 \ll k$ și operatorul AND pe biți.

**Problema 10.** Calculați manual și apoi verificați în C++:

```
int a = 5, b = 7, c = 2;
int x = a + b * c;
int y = (a + b) * c;
int z = a > b || b % c == 1;
```

**Problema 11.** Definiți variabile cu `auto` pentru un întreg, un număr real, un caracter și o valoare logică. Folosiți `decltype` pentru a declara câte o a doua variabilă de același tip.

**Verificarea soluțiilor**

Pentru fiecare problemă se recomandă următoarea ordine: identificarea tipurilor și valorilor implicate, scrierea expresiei, analiza precedenței și a conversiilor, estimarea rezultatului, implementarea și verificarea prin rulare.

*Noțiunile urmărite: variabile, identificatori, literale, tipuri fundamentale, specificatori, `sizeof`, expresii, operatori, precedență, conversii și atribuiri.*